

Interactuar con un sistema operativo UNIX (XV6)

INTEGRANTES:

- **Jhon William Mamani Condori 2020-119018**
- **Juan Luis Mamani Mullo 2021-119123**

Contenido

- 01 | Introducción
- 02 | Objetivo
- 03 | Modificaciones
- 04 | Resultados
- 05 | Conclusiones



Introducción

Los sistemas operativos son fundamentales porque administran recursos como CPU, memoria y dispositivos, permitiendo que varios procesos se ejecuten de forma concurrente con seguridad y eficiencia. Para reforzar estos conceptos en la práctica, se utiliza XV6, una versión educativa y simplificada de UNIX que facilita el análisis y la modificación del kernel. En este trabajo se amplía XV6 mediante la instrumentación de llamadas al sistema, la creación de nuevos comandos de usuario y mecanismos de monitoreo interno, con el fin de observar su comportamiento y relacionar los resultados con los temas de la Unidad II del curso.



Objetivo

- Registrar e instrumentar las syscalls de XV6 para monitorear su ejecución y parámetros.
- Implementar comandos de usuario para inspeccionar el estado del sistema y de los procesos.
- Crear un contador de invocaciones por syscall, accesible desde espacio de usuario.
- Realizar pruebas prácticas para evaluar el comportamiento del sistema y analizar resultados.
- Vincular los resultados con los conceptos de planificación, memoria y gestión de procesos de la Unidad II.

Modificaciones realizadas en XV6

Instrumentación del kernel

- Se añadió una bandera `trace_syscalls` para activar/desactivar la traza de syscalls.
- Se modificó `syscall()` para imprimir el nombre de cada llamada al sistema.
- Se creó la syscall `trace()` y el comando de usuario `trace` para controlar la traza.

Nuevos comandos de monitoreo

- `uptime2`: muestra ticks desde el arranque, procesos RUNNING/RUNNABLE/SLEEPING y cambios de contexto.
- `psmen`: lista procesos con su PID, estado, memoria y nombre usando la nueva syscall `getps()`.
- Programa `spin`: genera un proceso siempre RUNNABLE para probar la planificación.

Contador de llamadas al sistema

- Se agregó el arreglo `syscall_count[]` en el kernel para contar invocaciones de cada syscall.
- Se creó la syscall `getsyscount()` para leer esos contadores desde usuario.
- Comando `syscount`: muestra el número de veces que se ha ejecutado cada llamada al sistema o una en específico.

creados

- `trace.c`

Comando de usuario `trace` para activar/desactivar la traza de syscalls.

- `uptime2.c`

Comando `uptime2` que muestra ticks, procesos por estado y cambios de contexto.

- `psmen.c`

Comando `psmen` que lista procesos (PID, estado, memoria, nombre).

- `spin.c`

Programa `spin` que se queda en bucle infinito para generar un proceso RUNNABLE.

- `syscount.c`

Comando `syscount` que muestra cuántas veces se ejecutó cada syscall.

Resultados 1

Ejecución de trace y listado de syscalls

```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ _
```

```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ trace 1
[syscall] trace(1, 0, 0)
[syscall] wait(0, 16364, 12248)
[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
```



```
unjbg@unjbg-VirtualBox: ~/Escritorio/git/xv6
t 58
init: starting sh
$ trace 1
[syscall] trace(1, 0, 0)
[syscall] wait(0, 16364, 12248)
$[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
ls
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] fork(0, 16364, 12248)
[syscall] sbrk(32768, 0, 16072)
[syscall] exec(1, 12276, 12284)
[syscall] open(2688, 0, 0)
[syscall] fstat(3, 11644, 0)
[syscall] read(3, 11628, 16)
[syscall] open(11664, 0, 0)
[syscall] fstat(4, 11644, 0)
[syscall] close(4, 11644, 0)
.[syscall] write(1, 11519, 1)
[syscall] write(1, 11519, 1)
[syscall] write(1, 11519, 1)
[syscall] write(1, 11519, 1)
```

```
QEMU
Machine View
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] read(3, 11628, 16)
[syscall] close(3, 11628, 16)
[syscall] wait(0, 16364, 12248)
$[syscall] write(2, 16287, 1)
[syscall] write(2, 16287, 1)
trace 0
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] read(0, 16287, 1)
[syscall] fork(0, 16364, 12248)
[syscall] sbrk(32768, 0, 16072)
[syscall] exec(2, 12264, 12280)
$ _
```

Resultados 2

Ejecución del uptime2 y psmen

```
QEMU
Machine View
SeaBIOS (version 1.15.0-1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ uptime2
Tiempo desde arranque: 501 ticks (5 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 38
$
```

```
QEMU - Press Ctrl+Alt+G to release grab
Machine View
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1FF8B590+1FECB590 CA00

Booting from Hard Disk...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ uptime2
Tiempo desde arranque: 501 ticks (5 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 38
$ spin &
$ uptime2
Tiempo desde arranque: 41052 ticks (410 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 1
Procesos SLEEPING: 2
Cambios de contexto: 703
$ _
```

Resultados 3

Eliminación de un proceso

```
Cambios de contexto: 703
$ psmen
PID      Estado      Mem(Bytes)  Nombre
1        SLEEPING    12288       init
2        SLEEPING    16384       sh
7        RUNNING    12288       psmen
5        RUNNABLE    12288       spin
$
```

```
$ kill 5
zombie!
$ uptime2
Tiempo desde arranque: 50755 ticks (507 segundos)
Procesos RUNNING : 1
Procesos RUNNABLE: 0
Procesos SLEEPING: 2
Cambios de contexto: 9840
$ psmen
PID      Estado      Mem(Bytes)  Nombre
1        SLEEPING    12288       init
2        SLEEPING    16384       sh
10       RUNNING    12288       psmen
$
```

Ejecución del syscount

```
$ syscount
Listado de invocaciones:
Syscall 1: 14 veces
Syscall 2: 11 veces
Syscall 3: 14 veces
Syscall 5: 102 veces
Syscall 6: 1 veces
Syscall 7: 14 veces
Syscall 10: 2 veces
Syscall 12: 12 veces
Syscall 15: 2 veces
Syscall 16: 1308 veces
Syscall 21: 1 veces
Syscall 23: 3 veces
Syscall 24: 2 veces
Syscall 25: 5 veces
Syscall 32: 70205 veces
Syscall 56: 548789 veces
Syscall 58: 548798 veces
Syscall 60: 548807 veces
Syscall 62: 548816 veces
$ _
```

QEMU - Press Ctrl+Alt+G to release grab

Machine View

```
$ syscount 62
Syscall 62 fue invocada 548816 veces
```


Conclusiones

Las modificaciones realizadas en XV6 permitieron entender de forma práctica cómo el sistema operativo gestiona los procesos y las llamadas al sistema. A través de los comandos implementados se observó el funcionamiento interno del kernel, los estados de los procesos y la interacción entre usuario y kernel mediante trampas. Este trabajo reforzó los conceptos centrales de la Unidad II y brindó una visión clara del ciclo de vida de un proceso y la estructura interna del sistema operativo.



Gracias